LAB 5 — Internet Edge + ISP Serial WAN + Week03 Microservices

0. Members

Student ID	Name	Role
673380416-3	นายพัทธดนย์ คำนัน	**Network Architects**
673380428-6	นายสิทธิโชค มุขนาค	**IaC/DevOps**
673380583-4	นายณัฐภัทร ฉ่ำตะคุ	**QA/Test**
673380606-8	นายสรวิศ สุคงเจริญ	**QA/Test**
673380608-4	นางสาวอมลวรรณ พิมพิชัย	**Network Architects**

1. Lab Objective

วัตถุประสงค์ของ Lab นี้คือการจำลองสถาปัตยกรรมเครือข่ายระดับองค์กร โดยใช้ Docker container เป็นอุปกรณ์เครือข่ายและเซิร์ฟเวอร์ เพื่อทดสอบการทำงานของ

- Internet Edge Router (R1)
- ISP WAN Serial Link
- Branch Router (R2)
- Microservices deployment บน ServerA และ ServerB
- Network Address Translation (NAT)

Lab นี้ขยายจาก Lab 4 โดยเพิ่ม WAN topology และ Internet Edge architecture เพื่อจำลองระบบ distributed microservices ที่ทำงานข้ามเครือข่ายจริง

โดยโค้ดของ Lab 4 นักศึกษาได้จัดทำและอัพโหลดขึ้น GitHub เรียบร้อยแล้ว ท่านอาจารย์สามารถดูได้ผ่าน repo ของนักศึกษา ตามลิงค์ที่ได้แนบไว้นี้

GitHub link: https://github.com/ms584/CP352005-Computer-Networks-Laboratory/tree/main/network%20lab%205

2. Network Topology

โครงสร้างเครือข่ายที่ใช้ใน Lab นี้ประกอบด้วย

```
Internet
   |
   | DHCP
   |
R1 (Edge Router)
10.255.0.2
   |
ISP WAN
100.10.10.0/29
   |
R2 (Branch Router)
   |
----------------------------
|                          |
LAN A                    LAN B
192.168.10.0/24          192.168.20.0/24
|                          |
ServerA                  ServerB
ClientA                  ClientB
```

บทบาทของอุปกรณ์

Device	**Role**
R1	Internet Edge Router + NAT Gateway
R2	Branch Router
ServerA	Microservices Stack A
ServerB	Microservices Stack B
ClientA	Testing client

3. Address Table

Device	Interface	IP Address	Role
R1	eth0	10.255.0.2	Internet
R1	eth1	100.10.10.1	ISP WAN
R1	eth2	192.168.10.1	LAN A
R2	eth0	100.10.10.2	WAN
R2	eth1	192.168.20.1	LAN B
ServerA	eth0	192.168.10.10	Microservices
ServerB	eth0	192.168.20.10	Microservices
ClientA	eth0	192.168.10.50	Client

4. Infrastructure Deployment

เริ่ม container infrastructure ด้วย Docker Compose

docker compose down -v, docker compose up -d

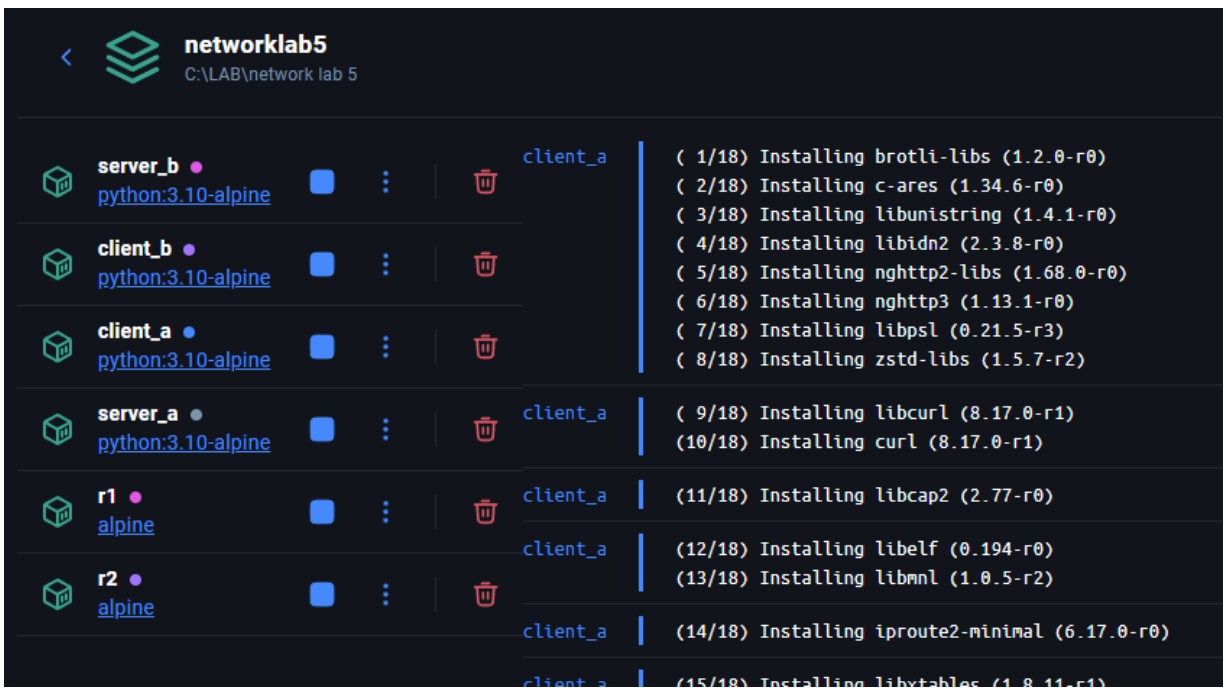
```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 5
$ docker compose down -v

Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 5
$ docker compose up -d
[+] Running 10/10
 ✓ Network networklab5_lan_b_net     Created
 ✓ Network networklab5_lan_a_net     Created
 ✓ Network networklab5_internet_net  Created
 ✓ Network networklab5_isp_net       Created
 ✓ Container ClientB                 Started
 ✓ Container R2                      Started
 ✓ Container ServerB                 Started
 ✓ Container ServerA                 Started
 ✓ Container ClientA                 Started
 ✓ Container R1                      Started
```

ตรวจสอบ container: docker ps

ผลลัพธ์

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 5
$ docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED         STATUS         PORTS     NAMES
ee17e95108e8   python:3.10-alpine   "sh -c ' apk add --n…"   5 seconds ago   Up 3 seconds             ServerB
71ebc6336f46   python:3.10-alpine   "sh -c ' apk add --n…"   5 seconds ago   Up 3 seconds             ClientB
753006a84279   python:3.10-alpine   "sh -c ' apk add --n…"   5 seconds ago   Up 3 seconds             ClientA
fc307832655b   python:3.10-alpine   "sh -c ' apk add --n…"   5 seconds ago   Up 4 seconds             ServerA
aa45fde8e7e0   alpine               "sh -c ' apk add --n…"   5 seconds ago   Up 3 seconds             R1
d4fa8d7380d0   alpine               "sh -c ' apk add --n…"   5 seconds ago   Up 3 seconds             R2
```

แสดงว่า network infrastructure ถูกสร้างสำเร็จ



5. Deploy Microservices

Microservices ถูก deploy บน ServerA และ ServerB

docker exec -d ServerA sh -c "python /automation/start_services.py"

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 5
$ docker exec -d ServerA sh -c "python /automation/start_services.py"
```

docker exec -d ServerB sh -c "python /automation/start_services.py"

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 5
$ docker exec -d ServerB sh -c "python /automation/start_services.py"
```

ตรวจสอบ process

docker exec ServerA ps

docker exec ServerB ps

ผลลัพธ์

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 5
$ docker exec ServerA ps
PID   USER     TIME  COMMAND
    1 root      0:00 tail -f /dev/null
   14 root      0:00 python /automation/start_services.py
   24 root      0:00 ps

Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 5
$ docker exec ServerB ps
PID   USER     TIME  COMMAND
    1 root      0:00 tail -f /dev/null
   14 root      0:00 python /automation/start_services.py
   24 root      0:00 ps
```

แสดงว่า microservice stack ทำงานแล้ว

6. Microservice Port Verification

ตรวจสอบ port ที่เปิด

docker exec ServerA ss -tuln

docker exec ServerB ss -tuln

ผลลัพธ์

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 5
$ docker exec ServerA ss -tuln
Netid State  Recv-Q Send-Q Local Address:Port  Peer Address:Port
udp   UNCONN 0      0         127.0.0.11:57547      0.0.0.0:*
tcp   LISTEN 0      5            0.0.0.0:8002       0.0.0.0:*
tcp   LISTEN 0      5            0.0.0.0:8000       0.0.0.0:*
tcp   LISTEN 0      5            0.0.0.0:8001       0.0.0.0:*
tcp   LISTEN 0      4096      127.0.0.11:44611      0.0.0.0:*
tcp   LISTEN 0      5            0.0.0.0:9000       0.0.0.0:*

Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 5
$ docker exec ServerB ss -tuln
Netid State  Recv-Q Send-Q Local Address:Port  Peer Address:Port
udp   UNCONN 0      0         127.0.0.11:33602      0.0.0.0:*
tcp   LISTEN 0      4096      127.0.0.11:34537      0.0.0.0:*
tcp   LISTEN 0      5            0.0.0.0:9000       0.0.0.0:*
tcp   LISTEN 0      5            0.0.0.0:8002       0.0.0.0:*
tcp   LISTEN 0      5            0.0.0.0:8001       0.0.0.0:*
tcp   LISTEN 0      5            0.0.0.0:8000       0.0.0.0:*
```

Port	Service
8000	Upload Service
8001	Processing Service
8002	AI Service
9000	Gateway Service

แสดงว่า microservices ถูก deploy ครบ

7. Network Connectivity Test

ทดสอบการเชื่อมต่อระหว่าง LAN

docker exec -it ClientA ping -c 3 192.168.20.10

ผลลัพธ์

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 5
$ docker exec -it ClientA ping -c 3 192.168.20.10
PING 192.168.20.10 (192.168.20.10): 56 data bytes
64 bytes from 192.168.20.10: seq=0 ttl=62 time=0.117 ms
64 bytes from 192.168.20.10: seq=1 ttl=62 time=0.085 ms
64 bytes from 192.168.20.10: seq=2 ttl=62 time=0.093 ms

--- 192.168.20.10 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.085/0.098/0.117 ms
```

แสดงว่า routing ระหว่าง LAN A และ LAN B ทำงานถูกต้อง

เส้นทาง packet

```
ClientA
→ R1
→ ISP WAN
→ R2
→ ServerB
```

8. Cross-Site Microservice Test

Upload Service: curl http://192.168.20.10:8000/health

ผลลัพธ์

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 5
$ docker exec -it ClientA sh -c "curl -v http://192.168.20.10:8000/health; echo"
*   Trying 192.168.20.10:8000...
* Established connection to 192.168.20.10 (192.168.20.10 port 8000) from 192.168.10.50 port 32986
* using HTTP/1.x
> GET /health HTTP/1.1
> Host: 192.168.20.10:8000
> User-Agent: curl/8.17.0
> Accept: */*
>
* Request completely sent off
* HTTP 1.0, assume close after body
< HTTP/1.0 200 OK
< Server: SimpleHTTP/0.6 Python/3.10.20
< Date: Fri, 06 Mar 2026 07:43:02 GMT
< Content-type: application/json
<
* shutting down connection #0
```

Gateway Workflow

curl -X POST http://192.168.20.10:9000/process-file

ผลลัพธ์

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 5
$ docker exec -it ClientA sh -c "curl -v -X POST http://192.168.20.10:9000/process-file; echo"
*   Trying 192.168.20.10:9000...
* Established connection to 192.168.20.10 (192.168.20.10 port 9000) from 192.168.10.50 port 45650
* using HTTP/1.x
> POST /process-file HTTP/1.1
> Host: 192.168.20.10:9000
> User-Agent: curl/8.17.0
> Accept: */*
>
* Request completely sent off
* HTTP 1.0, assume close after body
< HTTP/1.0 200 OK
< Server: SimpleHTTP/0.6 Python/3.10.20
< Date: Fri, 06 Mar 2026 07:43:09 GMT
< Content-type: application/json
<
* shutting down connection #0
```

แสดงว่า ClientA สามารถเรียก microservices บน ServerB ผ่าน WAN ได้สำเร็จ

9. Internet Connectivity Test

ทดสอบจาก ServerA

docker exec -it ServerA ping -c 3 8.8.8.8

ผลลัพธ์

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 5
$ docker exec -it ServerA ping -c 3 8.8.8.8
PING 8.8.8.8 (8.8.8.8): 56 data bytes
64 bytes from 8.8.8.8: seq=0 ttl=62 time=13.740 ms
64 bytes from 8.8.8.8: seq=1 ttl=62 time=14.132 ms
64 bytes from 8.8.8.8: seq=2 ttl=62 time=14.114 ms

--- 8.8.8.8 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 13.740/13.995/14.132 ms
```

แสดงว่า ServerA สามารถออก Internet ผ่าน R1 ได้

10. Router Verification

R1 Routing Table

docker exec R1 ip route

ผลลัพธ์: default via 10.255.0.1, 192.168.20.0/24 via 100.10.10.2

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 5
$ docker exec R1 ip route
default via 10.255.0.1 dev eth0
10.255.0.0/24 dev eth0 proto kernel scope link src 10.255.0.2
100.10.10.0/29 dev eth1 proto kernel scope link src 100.10.10.1
192.168.10.0/24 dev eth2 proto kernel scope link src 192.168.10.1
192.168.20.0/24 via 100.10.10.2 dev eth1
```

R2 Routing Table

docker exec R2 ip route

ผลลัพธ์: default via 100.10.10.1, 192.168.10.0/24 via 100.10.10.1

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 5
$ docker exec R2 ip route
default via 100.10.10.1 dev eth0
100.10.10.0/29 dev eth0 proto kernel scope link src 100.10.10.2
192.168.10.0/24 via 100.10.10.1 dev eth0
192.168.20.0/24 dev eth1 proto kernel scope link src 192.168.20.1
```

11. NAT Verification

ตรวจ NAT rule: docker exec R1 iptables -t nat -L -n

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 5
$ docker exec R1 iptables -t nat -L -n
Chain PREROUTING (policy ACCEPT)
target     prot opt source               destination

Chain INPUT (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
DOCKER_OUTPUT  all  --  0.0.0.0/0            127.0.0.11

Chain POSTROUTING (policy ACCEPT)
target     prot opt source               destination
DOCKER_POSTROUTING  all  --  0.0.0.0/0            127.0.0.11
MASQUERADE  all  --  0.0.0.0/0            0.0.0.0/0

Chain DOCKER_OUTPUT (1 references)
target     prot opt source               destination
DNAT       tcp  --  0.0.0.0/0            127.0.0.11           tcp dpt:53 to:127.0.0.11:46549
DNAT       udp  --  0.0.0.0/0            127.0.0.11           udp dpt:53 to:127.0.0.11:43359

Chain DOCKER_POSTROUTING (1 references)
target     prot opt source               destination
SNAT       tcp  --  127.0.0.11           0.0.0.0/0            tcp spt:46549 to::53
SNAT       udp  --  127.0.0.11           0.0.0.0/0            udp spt:43359 to::53
```

พบ rule

MASQUERADE all -- 0.0.0.0/0

แสดงว่า R1 ทำ NAT overload

Connection Tracking

docker exec R1 cat /proc/net/nf_conntrack | grep 8.8.8.8

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 5
$ docker exec R1 cat /proc/net/nf_conntrack | grep 8.8.8.8
ipv4     2 icmp     1 29 src=192.168.10.10 dst=8.8.8.8 type=8 code=0 id=42 src=8.8.8.8 dst=10.255.0.2 type=0 code=0 id=42 mark=0 zone=0 use=2
```

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 5
$ docker exec -it ServerA ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8): 56 data bytes
64 bytes from 8.8.8.8: seq=0 ttl=62 time=29.727 ms
64 bytes from 8.8.8.8: seq=1 ttl=62 time=13.906 ms
64 bytes from 8.8.8.8: seq=2 ttl=62 time=14.650 ms
64 bytes from 8.8.8.8: seq=3 ttl=62 time=14.086 ms
64 bytes from 8.8.8.8: seq=4 ttl=62 time=13.827 ms
```

ผลลัพธ์: src=192.168.10.10 dst=8.8.8.8, src=8.8.8.8 dst=10.255.0.2

แสดงว่า NAT translation เกิดขึ้นจริง

12. Architectural Significance

Lab นี้จำลองสถาปัตยกรรมจริงของระบบ distributed services

แนวคิดที่แสดงใน Lab

Concept	**Implementation**
Enterprise Edge Router	R1
Branch Network	R2
ISP WAN	100.10.10.0/29
Service Distribution	ServerA / ServerB
NAT Boundary	R1
Cross-Site Services	Gateway API

13. Conclusion

Lab 5 ประสบความสำเร็จในการสร้าง distributed microservice architecture ที่ทำงานข้าม WAN network

ผลลัพธ์สำคัญของ Lab

- Routing ระหว่าง LAN ทำงานถูกต้อง
- Cross-site microservice access สำเร็จ
- Internet connectivity ผ่าน NAT ทำงาน
- Microservices stack ทำงานครบทุก service

Lab นี้แสดงให้เห็นการทำงานของ

- Internet Edge Routing
- WAN connectivity
- Network segmentation
- Distributed service architecture

ซึ่งเป็นพื้นฐานของระบบ cloud และ enterprise networking